

LeafOS Codebase Overview

Source Tree and Responsibility Map

LeafOS

Purpose

This document is a printable map of the current LeafOS codebase. It separates packaging surfaces, source implementation, model acquisition, hardware monitoring, and generated evidence. The current project path is `leafctl live PATH`: read-only intake, a typed version-2 run, resident supervision, provider proposals, CPU-authoritative execution and validation, checkpointing, reporting, and bounded follow-up work. It should be read beside `docs/DOCUMENTATION_MAP.md`, `docs/ARCHITECTURE.md`, and the taskpack roadmap.

Top-Level Shape

```
LeafOS/  
|-- PowerShell-Version/      Windows-friendly command surface  
|-- Bash-Version/           Bash / WSL / Linux command surface  
|-- ProjectLeaf/  
|   |-- leafos_taskpack/     primary CLI, graph, agent, node, provider  
|   runtime  
|   |-- leaf_model_installer/ plan-first local model acquisition package  
|   '-- floweros_hardware_monitor/  
|                               terminal hardware/GPU/fan telemetry utility  
|-- README.md               root orientation  
|-- USAGE.md                daily command guide  
|-- STAGE.md                 evidence and promotion plane  
|-- leafos.root.json         root layout and runtime policy  
'-- VERSION                  root packaging label
```

Responsibility Boundaries

Area	Responsibility
Root	Defines the two-surface package layout and points operators to the real command surfaces.
PowerShell surface	Windows-first wrappers, install scripts, real-model checks, one-shot bundles, and the llama.cpp provider launcher.
Bash surface	Bash-first wrappers for the same high-level workflows.
Taskpack	The active LeafOS runtime: live-project inlet, resident scheduler, resource governor, providers, CPU executor, TUI, telemetry, compatibility graph/node tools, and reports.
Model installer	Catalog, plan, resolve, apply, resume, and verify model weights. This owns the download boundary.
Hardware monitor	Separate telemetry utility for CPU, memory, storage, network, battery, temperature, GPU, and fan data.

Reports and runs	Evidence artifacts. They are useful for audit, but they should not be treated as source.
------------------	--

Taskpack Subsystems

Path	Role
<code>bin/</code>	Main command dispatch and user-facing CLI entrypoints.
<code>core/agent/</code>	Task creation, plans, dry-runs, graph loop actions, repair injection, and run manifests.
<code>core/graph/</code>	Graph state, dependency readiness, graph validation, and graph printing.
<code>core/model/</code>	Brain and coder wrappers that call the provider layer and write artifacts.
<code>core/python/</code>	Version-2 inlet, live-project intake, resident supervisor, resource governor, telemetry, work orders, and CPU execution lifecycle.
<code>core/providers/</code>	Provider adapters. Production resident runs use explicit local or configured providers; mock adapters are test-only.
<code>core/patch/</code>	Patch validation, preview, and explicit application boundary.
<code>core/verify/</code>	Completion gate and verification command handling.
<code>core/ui/tui/</code> , <code>core/tui/</code>	Python and native renderers over one normalized snapshot and typed-control bridge.
<code>core/node/</code>	Optional file-backed SSH node receiver and task state machine.
<code>core/remote/</code>	Distributor-side local/SSH transport.
<code>core/net/</code>	LAN discovery, local serving, and bundle download helpers.
<code>core/runtime/</code>	Runtime role policy, model/persona selection, and worker routing.
<code>core/ui/</code> , <code>core/web/</code>	Read-only local views for chat, dashboard, and web/runtime state.
<code>config/</code>	Runtime JSON, provider defaults, action schema, glyphs, loaders, and brand config.
<code>tests/</code>	167 discovered Python contracts plus shell, native TUI, Catan2, graph, provider, node, runtime, orchestration, and compatibility gates.

What Is Real Today

- `leafctl live` PATH inventories an existing project or creates a minimal seed, starts or reattaches a persistent version-2 run, starts the resident supervisor, and opens the TUI.
- Instructions from plain text, typed JSON tasks, and work orders converge on one priority/dependency queue and the lifecycle inspect, plan, approve, execute, validate, checkpoint, and report.
- The local `llama.cpp` Vulkan provider supplies schema-constrained proposals. CPU policy owns paths, commands, approval, execution, validation, repair, and evidence.
- The resource governor uses foreground activity, responsiveness, CPU, GPU, RAM, VRAM, temperature, provider health, and budgets to admit or defer useful work.
- Python and native TUI renderers consume one normalized snapshot and send bounded authenticated controls without editing run or project files.
- Universal telemetry, LMEM controls, event cursors, worker/resident leases, checkpoint hashes, and reports support inspection and recovery.

- The retained real four-minute Catan2 resident profile passed with one completed resident-generated improvement and no blocked work.
- The older task spine, graph/action runner, one-shot bundles, and SSH node tools remain compatibility or optional subsystems.

Open Acceptance And Compatibility Work

- Run and retain the real eight-minute and 64-minute WO-038 Catan2 resident profiles before declaring the complete soak matrix passed.
- Replace remaining gated mock-provider tests with equivalent recorded or live llama.cpp fixtures; preserve historical telemetry readability.
- Materialize separate main, scheduler, and coder model roles while preserving proposal-only provider authority.
- Expand TUI task composition and search through typed controls, never direct renderer mutation.
- Keep optional node, graph, and one-shot documentation explicit about their scope.

Operating Doctrine

```
Files are the state carrier.  
The typed inlet is the execution authority.  
The CLI and TUI are control surfaces.  
JSON and JSONL are the interchange contracts.  
SSH is the node transport.  
llama.cpp is a provider endpoint, not the authority.  
The CPU validates, gates, writes, and records evidence.  
Useful-work targets adapt to foreground and hardware pressure.
```